RESERVAS DE HOTEL

Automatización del proceso de solicitud, validación y consulta de disponibilidad

INTEGRANTES

Ana Sofia Ricardo

Mariana Valencia

Juan Martin Garcia

Luis Fernando Velazques

Presentación técnica y evidencias de pruebas

Contenido

1. Proyecto y beneficiarios
2. Problema y contexto
3. Proceso manual y dificultades
4. Objetivo y alcance de esta versión
5. Solución automatizada y arquitectura
6. Recorrido de una solicitud completa
7. Integraciones
8. Organización de los workflows
9. Validaciones y manejo de errores
10. Pruebas, resultados y evidencias
11. Limitaciones y mejoras futuras

> **Nota de alcance:** el material de apoyo también indica que la respuesta `503` para fallas técnicas sigue pendiente en el workflow principal y recomienda rotar las claves expuestas durante las pruebas. Conviene revisar estos puntos antes de una demostración pública.

12. Repositorio y estado de la entrega
Glosario

> **Nota sobre el alcance de la evidencia**
>
> El ZIP contiene workflows de n8n, bitácoras de pruebas y capturas de ejecuciones manuales. Las pruebas descritas no demuestran por sí solas una transacción completa realizada por un cliente real a través del `Webhook`. Por eso, el documento distingue entre resultados observados en nodos, funcionalidades configuradas y aspectos aún por comprobar.

1. Proyecto y beneficiarios

El proyecto automatiza partes del proceso de reservas de un hotel mediante workflows de n8n y consultas a Baserow. La solución recibe una solicitud, valida y normaliza sus datos, consulta habitaciones y reservas existentes, determina si hay disponibilidad o si la solicitud está duplicada, y devuelve un resultado al sistema que inició la petición.

Los beneficiarios previstos son el personal que administra las reservas y las personas que solicitan una habitación. La automatización busca reducir tareas repetitivas de revisión, evitar asignaciones sobre habitaciones ocupadas y detectar solicitudes duplicadas. Estos son beneficios esperados de la lógica implementada; el material entregado no contiene mediciones de ahorro de tiempo ni de reducción porcentual de errores.

> **Cómo funciona, en pocas palabras**
>
> Para quien no conoce n8n: un *workflow* es una cadena de pasos, llamados *nodos*, que se ejecutan automáticamente; cada nodo hace una tarea concreta (recibir datos, validarlos, consultar una tabla o responder). Así se recorre una solicitud según el diseño documentado:
>
> 1. Quien solicita la habitación, o un formulario, envía una petición `POST` en formato JSON al `Webhook` de n8n con nombre, correo, fechas, número de huéspedes y tipo de habitación.
> 2. n8n limpia y valida esos datos. Si son incorrectos, responde con un error `400`.
> 3. Si son válidos, consulta en Baserow las habitaciones del tipo pedido y las reservas confirmadas para decidir si hay disponibilidad o si la solicitud ya existe.
> 4. Si hay habitación, el diseño prevé guardar la reserva en Baserow y responder `200` con un mensaje de confirmación; si no la hay, responde `409`.
> 5. Si una ejecución falla por un problema técnico, un workflow aparte está pensado para registrar la incidencia.
>
> Los pasos 4 y 5 describen el diseño de los workflows; la prueba integral de punta a punta no está demostrada (ver sección 10).

Elemento	Descripción
Nombre del proyecto	Automatización de reservas de hotel
Orquestación	n8n, mediante un workflow principal y subworkflows
Persistencia y consulta de datos	Baserow, para habitaciones, reservas e incidencias
Entrada principal	Petición HTTP `POST` al endpoint `/webhook/reserva-hotel`
Equipo	Ana Sofia Ricardo; Mariana Valencia; Juan Martin Garcia; Luis Fernando Velazques

¿Qué se construyó?

- Workflow principal de reservas que recibe la solicitud y coordina los pasos.
- Subworkflow de preparación de solicitud, con limpieza, validación y cálculo de noches.
- Subworkflow de consulta de disponibilidad, que compara las fechas solicitadas con reservas confirmadas y detecta duplicados.
- Workflow de manejo de errores, pensado para registrar información técnica de incidentes en Baserow.
- Un puente HTTP local para un laboratorio de integración con Google Forms, n8n local y una ruta permitida específica; el ZIP incluye su código, pero no una evidencia de despliegue público.

El puente local no es obligatorio en el recorrido: la secuencia de la sección 5 lo presenta como «puente opcional» entre el formulario y el `Webhook` de n8n.

2. Problema y contexto

Una reserva no consiste únicamente en recibir el nombre del huésped y las fechas. Antes de confirmarla, es necesario comprobar que la solicitud tenga información válida, que la fecha de salida sea posterior a la de entrada, que exista una habitación del tipo solicitado y que no haya una reserva confirmada que se cruce con el periodo pedido. También conviene detectar una solicitud repetida para no crear reservas duplicadas.

La bitácora de disponibilidad identifica un problema concreto del flujo original: podía ofrecer una habitación sin comprobar todas las reservas que ya ocupaban ese mismo periodo. En la prueba `D-02`, la habitación `102` estaba ocupada por reservas existentes y el subworkflow actualizado eligió la habitación `201`. Esta comparación sustenta la necesidad de consultar tanto las habitaciones como las reservas.

> **Problema que aborda el sistema**
>
> La revisión manual o una lógica que solo filtre por tipo de habitación puede pasar por alto cruces de fechas y solicitudes repetidas. La solución incorpora reglas explícitas para revisar estos casos antes de proponer una habitación.

3. Proceso manual y dificultades

El material recibido no describe formalmente un procedimiento manual real de un hotel específico. Por tanto, el siguiente esquema se presenta como el proceso operativo que la automatización pretende apoyar, no como una observación documentada de un hotel:

1. Recibir los datos de la persona y las fechas solicitadas.
2. Comprobar que el nombre, correo, tipo de habitación, fechas y número de huéspedes estén completos y tengan un formato válido.
3. Consultar las habitaciones del tipo solicitado.
4. Revisar las reservas confirmadas para detectar cruces con las fechas pedidas.
5. Comprobar si la misma solicitud ya existe.
6. Informar si hay disponibilidad o si se debe corregir la solicitud.
7. Cuando corresponda, registrar la reserva y devolver una respuesta al solicitante.

Las dificultades que la automatización busca reducir son los errores de digitación, las revisiones repetitivas, los cruces de fechas no detectados y la duplicación de solicitudes. No se dispone de un estudio de tiempos ni de una medición previa del proceso manual.

La tabla siguiente indica, según el diseño documentado, qué parte de la solución cubre cada paso. Es una correspondencia de diseño, no una evidencia de ejecución.

Paso	Dónde se automatiza
1. Recibir los datos	`Webhook` del workflow principal (**`POST`** `/webhook/reserva-hotel`)
2. Comprobar datos completos y formato	Subworkflow «Preparar solicitud»
3. Consultar habitaciones del tipo solicitado	Subworkflow «Consultar disponibilidad» (consulta a Baserow con **`GET`**)
4. Revisar cruces con reservas confirmadas	Subworkflow «Consultar disponibilidad»
5. Comprobar si la solicitud ya existe	Subworkflow «Consultar disponibilidad» (detección de duplicados)
6. Informar disponibilidad o corrección	Workflow principal: respuestas **200**, **400** y **409**
7. Registrar la reserva y responder	Workflow principal: nodos `Crear reserva` y `Confirmación`; el guardado integral no está demostrado

4. Objetivo y alcance de esta versión

Objetivo general

Automatizar la recepción y validación de solicitudes de reserva, la consulta de habitaciones y reservas en Baserow, la detección de conflictos de fechas o solicitudes duplicadas y la devolución de un resultado estructurado.

En la práctica, quien envía la solicitud recibe una respuesta HTTP: un **400** si los datos son inválidos, un **409** si no hay disponibilidad o, en la rama exitosa del diseño, un **200** con un mensaje de confirmación (diagrama de la sección 8). El envío de correos o notificaciones al huésped no está acreditado; ver «Fuera de lo que las evidencias demuestran».

Incluido en la versión documentada

- Validación del nombre, correo electrónico, fechas, número de huéspedes y tipo de habitación.
- Normalización de texto y correo electrónico, conversión del número de huéspedes y cálculo de noches.
- Consulta de habitaciones y reservas confirmadas en Baserow.
- Detección de cruces de fechas y de solicitudes duplicadas.
- Respuestas diferenciadas para datos inválidos y para falta de disponibilidad en el workflow principal.
- Preparación de un registro técnico de incidente y flujo previsto para guardar ese registro.
- Pruebas manuales de nodos con entradas simuladas y capturas de evidencia.

Fuera de lo que las evidencias demuestran

- No se demuestra una prueba integral real, desde un cliente externo hasta una reserva guardada y confirmada en Baserow.
- No se acredita el envío de correos, mensajes ni notificaciones al huésped.
- El manejador de errores registra el incidente; no repara ni reanuda automáticamente el workflow fallido.

- No hay datos suficientes para cuantificar ahorro de tiempo, impacto económico o reducción porcentual de errores.
- No se adjunta una URL de GitHub del repositorio.

5. Solución automatizada y arquitectura

La arquitectura se compone de un workflow principal y workflows especializados. El principal coordina la solicitud; los subworkflows encapsulan la validación y la consulta de disponibilidad; el workflow de errores recibe metadatos de ejecuciones fallidas.

Componente	Función
Cliente o formulario	Envía los campos de la solicitud como JSON por HTTP `POST`.
Puente local (laboratorio)	Restringe el acceso a una ruta y método permitidos, valida el tipo y formato JSON y reenvía la solicitud a n8n local.
n8n: Hotel (principal)	Recibe la petición, llama a los subworkflows y devuelve la respuesta.
Subworkflow: Preparar solicitud	Valida, limpia y normaliza los campos y calcula las noches.
Subworkflow: Consultar disponibilidad	Consulta habitaciones y reservas, detecta cruces y duplicados y devuelve una decisión.
Baserow	Proporciona tablas de habitaciones, reservas e incidencias.
Manejador de errores	Prepara y pretende guardar un registro técnico cuando una ejecución falla.

Para que cada pieza se entienda dentro de la solución, la siguiente tabla detalla qué información recibe, qué envía y por qué es necesaria. Todo proviene de las tablas y descripciones de este documento.

Componente	Qué recibe	Qué envía	Por qué es necesario
Cliente o formulario	Los datos que introduce la persona: nombre, correo, fechas, huéspedes y tipo de habitación.	JSON por **POST** con esos campos.	Es el origen de la solicitud de reserva.
Puente local (laboratorio)	Solicitud JSON **POST** a `/webhook/reserva-hotel`; cuerpo de hasta 10 KB.	La misma solicitud, reenviada a n8n local (`127.0.0.1:5678`).	Limita el acceso del laboratorio a una ruta específica. Es opcional en el recorrido.
n8n: Hotel (principal)	La solicitud, mediante el `Webhook`.	Llamadas a los subworkflows y la respuesta HTTP (**400**, **409** o **200**).	Coordina el proceso completo y decide la respuesta.
Subworkflow: Preparar solicitud	Los campos originales de la solicitud.	`valido`, `errores`, campos estandarizados y `Noches`.	Evita consultar el inventario con datos incorrectos o incompletos.
Subworkflow: Consultar disponibilidad	Tipo, entrada, salida, huéspedes y email.	`disponible`, `duplicada`, motivo, habitación y contadores de registros revisados.	Decide si hay habitación libre y evita duplicados o cruces de fechas.
Baserow	Consultas **GET** (habitaciones y reservas); **POST** previsto para reservas e incidencias.	Los registros de las tablas consultadas.	Guarda y expone el inventario, las reservas y las incidencias.
Manejador de errores	Metadatos de una ejecución fallida: workflow, ID y URL de ejecución, último nodo y mensaje del error.	Un registro de incidente preparado para Baserow.	Ayuda a localizar fallos técnicos; no los repara.

Diagrama general

La arquitectura se complementa con los diagramas específicos de los workflows de la sección 8. Representan la lógica funcional descrita en los archivos de apoyo y deben contrastarse con la versión final activa de cada workflow en n8n.

Secuencia	Qué ocurre
1. Entrada	Cliente / formulario → puente opcional → `Webhook` de n8n.
2. Validación	Workflow principal → «Preparar solicitud» → válido / inválido.
3. Consulta	Si los datos son válidos: «Consultar disponibilidad» → Baserow: habitaciones y reservas.
4. Decisión	Disponible, solicitud duplicada o sin disponibilidad.
5. Resultado	El workflow principal construye una respuesta para el solicitante.
6. Excepción técnica	Si falla una ejecución, el workflow de errores recibe los metadatos y prepara un registro de incidente.

> **Término técnico**
>
> `Webhook`: punto de entrada HTTP que permite que otro sistema envíe datos al workflow. En este proyecto, la ruta configurada es `/webhook/reserva-hotel` y el método es **`POST`**.

6. Recorrido de una solicitud

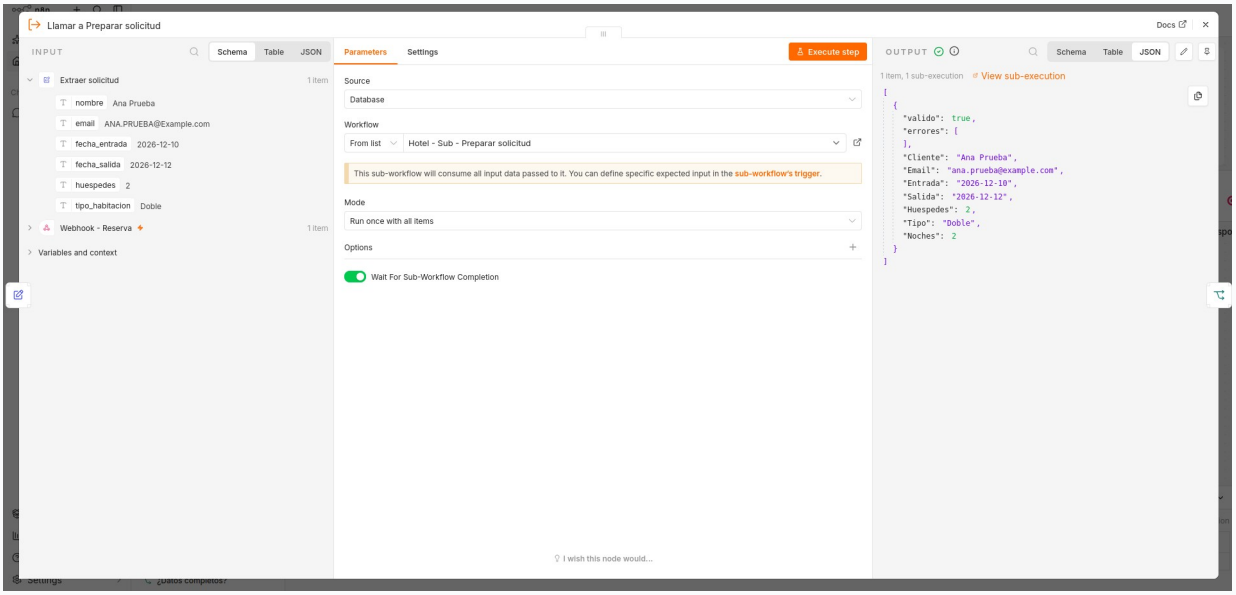
Ejemplo válido de prueba

La bitácora usa datos ficticios para demostrar la preparación de una solicitud. La entrada incluye espacios adicionales en el nombre, un correo en mayúsculas, fechas válidas y el número de huéspedes como texto.

Campo de entrada	Valor de ejemplo
`nombre`	Ana Prueba
`email`	ANA.PRUEBA@Example.com
`fecha_entrada`	2026-12-10
`fecha_salida`	2026-12-12
`huespedes`	2 (texto)
`tipo_habitacion`	Doble

Transformaciones observadas

Paso	Resultado
Limpieza del nombre	«Ana Prueba», sin espacios sobrantes.
Normalización del correo	`ana.prueba@example.com`, en minúsculas.
Conversión de huéspedes	El texto «2» se convierte en el número 2.
Cálculo de noches	2 noches entre el 10 y el 12 de diciembre de 2026.
Validación	`valido: true`; `errores`: lista vacía.



Evidencia: salida del nodo de preparación para una solicitud válida.

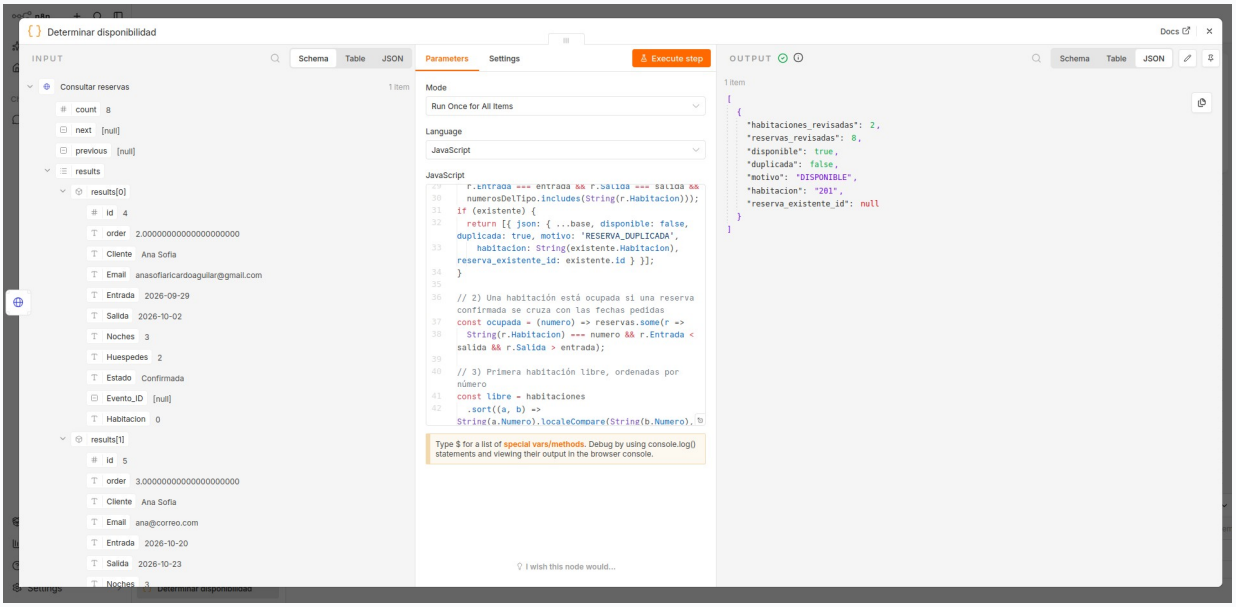
Qué significa este resultado. La entrada llegó con tres defectos típicos (espacios de más, correo en mayúsculas y huéspedes como texto) y la salida muestra los datos ya normalizados con `valido: true`. A nivel del nodo de preparación, esto indica que la limpieza, la conversión y el cálculo de noches se comportan como se espera con esta entrada ficticia.

Decisión de disponibilidad

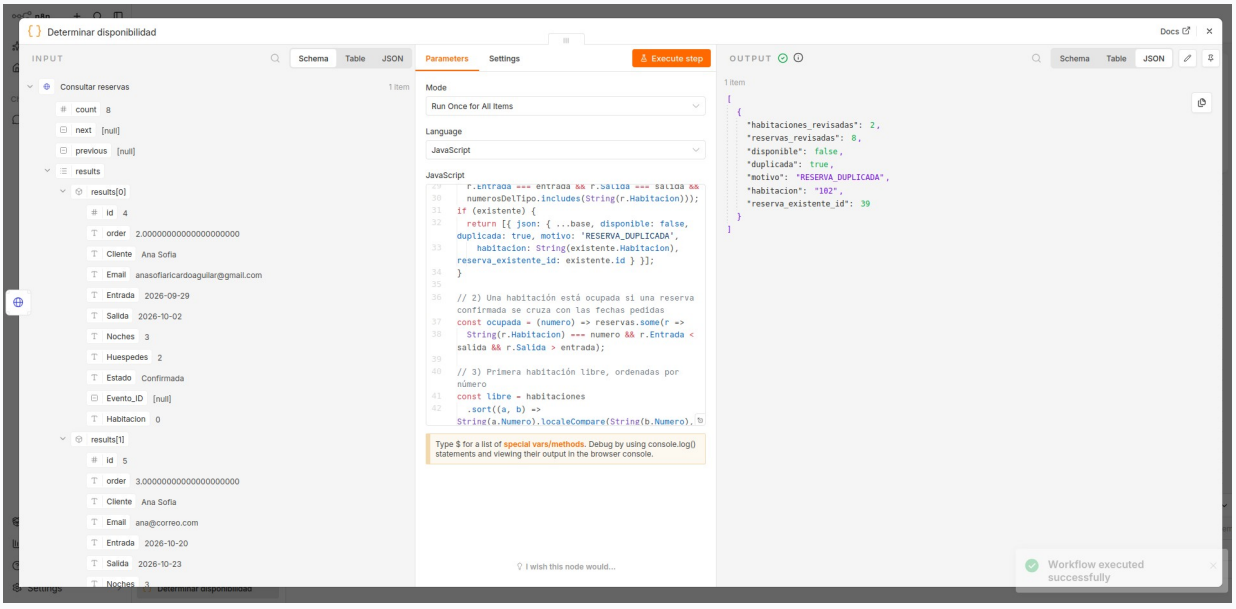
El subworkflow revisa las habitaciones del tipo solicitado y las reservas confirmadas. Una reserva ocupa una habitación si su fecha de entrada es anterior a la salida solicitada y su fecha de salida es posterior a la entrada solicitada. Así, una salida y una nueva entrada en el mismo día no se consideran un cruce.

Caso de prueba	Decisión observada
`D-01` · Fechas libres	Disponible; habitación 102.
`D-02` · Cruce de fechas	La 102 estaba ocupada; se eligió la 201.
`D-03` · Solicitud repetida	Duplicada: true; reserva existente ID 39; no propone crear otra reserva.
`D-04` · Tipo inexistente	Sin disponibilidad; no se encontró habitación del tipo solicitado.
`D-05` · Salida el mismo día	Disponible la habitación 102 porque la reserva anterior sale ese día.

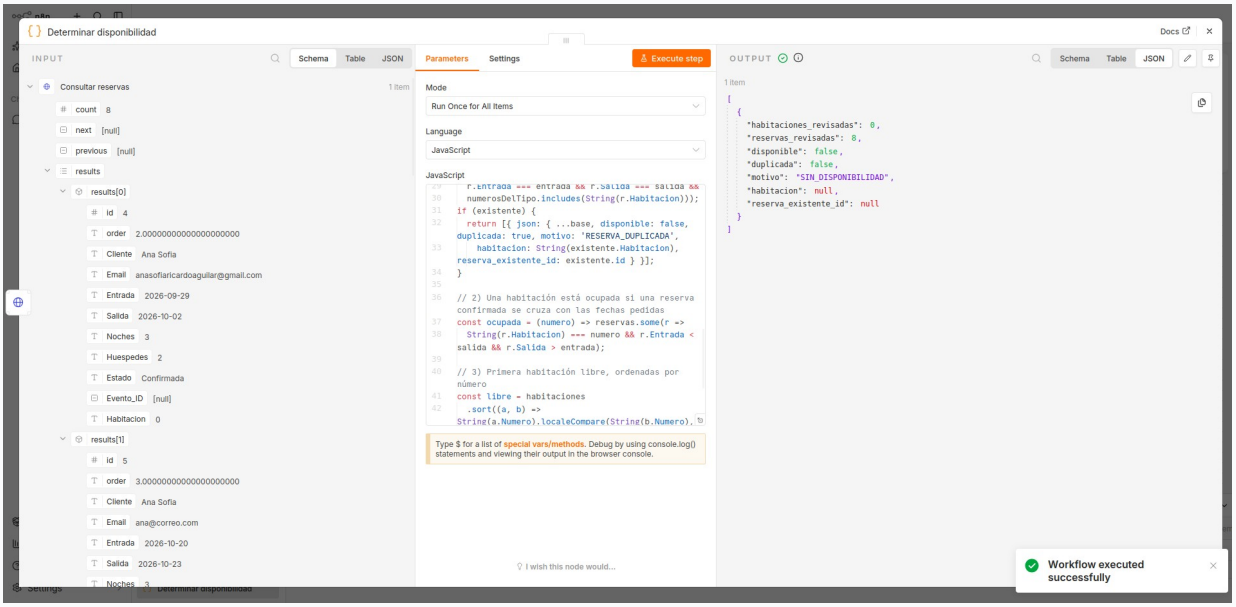
Cada caso busca comprobar una regla distinta: que se detecte una habitación libre (`D-01`), que se descarte la ocupada y se elija otra (`D-02`), que no se cree una reserva repetida (`D-03`), que se informe cuando el tipo pedido no existe (`D-04`) y que la regla de límites de fechas funcione (`D-05`). Las tres capturas siguientes corresponden a `D-02`, `D-03` y `D-04`.



Evidencia D-02: el resultado selecciona la habitación 201 al detectar ocupación de la 102.



Evidencia D-03: detección de una solicitud repetida.



Evidencia D-04: no se encuentra disponibilidad para el tipo solicitado.

Límite importante

Las pruebas del subworkflow de disponibilidad son ejecuciones manuales de solo lectura. Demuestran la decisión del nodo con los datos consultados, pero no prueban por sí solas que una solicitud HTTP real haya completado toda la operación ni que se haya creado una reserva.

7. Integraciones

La tabla describe cada conexión entre componentes: qué información viaja y para qué sirve. `GET` se usa para consultar datos y `POST` para enviarlos.

Integración	Información intercambiada	Propósito
Cliente / formulario → `Webhook`	JSON con nombre, email, fecha de entrada, fecha de salida, huéspedes y tipo de habitación.	Iniciar la automatización con una solicitud estructurada.
Workflow principal → subworkflow de preparación	Campos originales de la solicitud.	Validar y normalizar datos antes de consultar el inventario.
Workflow principal → subworkflow de disponibilidad	Tipo, entrada, salida, huéspedes y email.	Determinar disponibilidad y detectar duplicados.
n8n → Baserow (HTTP `GET`)	Consultas de habitaciones y reservas.	Leer inventario y reservas confirmadas.
n8n → Baserow (HTTP `POST` previsto en reserva)	Datos normalizados de una reserva.	Guardar una reserva; la prueba integral de escritura no está demostrada en las bitácoras adjuntas.
n8n → workflow de errores	Nombre del workflow, ID y URL de ejecución, último nodo y mensaje del error.	Preparar un registro técnico que ayude a localizar la incidencia.
Puente local → n8n local	Solicitud JSON reenviada solo a la ruta permitida.	Limitar el acceso del laboratorio a la ruta específica de reservas.

El puente de laboratorio escucha en `127.0.0.1:5681` y reenvía a n8n en `127.0.0.1:5678`. Según el código incluido, admite solo `POST` a `/webhook/reserva-hotel`, limita el cuerpo a 10 KB, establece un máximo simple de 30 peticiones por minuto y aplica un timeout de 15 segundos. Estas son características del código, no evidencia de que el servicio estuviera publicado o disponible externamente.

Como no todas las integraciones tienen el mismo grado de comprobación, la siguiente tabla separa lo probado de lo previsto o pendiente, según lo que el propio documento respalda.

Integración o servicio	Estado según la evidencia	Base en el documento
Cliente / formulario → `Webhook`	**Configurado, sin prueba real**	Ruta `/webhook/reserva-hotel` con `POST`; no hay prueba con una petición HTTP real.
Subworkflow «Preparar solicitud»	**Probado manualmente**	Pruebas `P-01` y `P-02` a nivel de nodo.
Subworkflow «Consultar disponibilidad» + Baserow `GET`	**Probado manualmente**	Pruebas `D-01` a `D-05`, ejecuciones de solo lectura.
Baserow `POST` (guardar reserva)	**Previsto; no demostrado**	Sin prueba integral de escritura; hay referencias antiguas por corregir en `Crear reserva` y `Confirmación` (sección 8).
Manejador de errores	**Parcial**	`M-01` observada; `M-02` y `M-03` pendientes; falta asociarlo desde la configuración del principal.
Puente local → n8n local	**Código incluido**	El ZIP incluye el código; no hay evidencia de despliegue público.
Google Calendar	**No integrado**	El documento indica que todavía no está integrado.

8. Organización de los workflows

Workflow	Responsabilidad y relación
Hotel (principal reservas)	Punto de entrada. Extrae la solicitud, llama a Preparar solicitud, y luego a Consultar disponibilidad. Devuelve el resultado HTTP según la rama tomada.
Hotel - Sub - Preparar solicitud	Recibe los campos originales, valida y normaliza; entrega `valido`, `errores`, campos estandarizados y `Noches`.
Hotel - Sub - Consultar disponibilidad	Consulta las habitaciones y las reservas; devuelve `disponible`, `duplicada`, `motivo`, habitación y contadores de registros revisados.
Hotel - Manejador de errores	Se activa ante fallos de workflows asociados. Prepara campos como fecha, workflow, ejecución, nodo, referencia de solicitud, mensaje y estado.

> **Qué significa subworkflow**
>
> Es un workflow auxiliar que se ejecuta desde otro workflow para concentrar una tarea concreta y reutilizable. Aquí se separan la preparación de datos y la consulta de disponibilidad del proceso principal.

La separación facilita mantener y probar cada responsabilidad de forma independiente. La documentación de disponibilidad también indica que la versión anterior podía ofrecer una habitación ocupada; el subworkflow nuevo añade una revisión explícita de las reservas antes de seleccionar una habitación.

Para ubicar cada flujo, esta ficha resume su punto de entrada, qué hace y qué devuelve, según las tablas y los diagramas que aparecen a continuación.

Workflow	Entrada y datos que recibe	Qué hace o decide	Qué devuelve
Hotel (principal)	Nodo `Webhook - Reserva`: recibe **`POST`** `/reserva-hotel` con los campos de la solicitud.	Extrae la solicitud y llama a «Preparar solicitud». Decide «¿Datos completos?» (No → **400**; Sí → «Consultar disponibilidad») y «¿Hay habitación?» (Sí → `Crear reserva` en Baserow y `Confirmación`; No → `Sin disponibilidad`).	Respuesta HTTP **400**, **409** o **200** según la rama.
Preparar solicitud	Los campos originales, enviados desde el principal.	Limpia espacios, normaliza el email, valida fechas y huéspedes, comprueba campos obligatorios y calcula el número de noches.	`valido`, `errores` y datos normalizados (entrada, salida, huéspedes, tipo, noches).
Consultar disponibilidad	Tipo, fechas, huéspedes y email.	Consulta habitaciones y reservas en Baserow; busca cruces de fechas, detecta duplicados y selecciona una habitación libre.	`disponible`, `duplicada`, motivo, habitación y datos de referencia.
Manejador de errores	Metadatos de una ejecución fallida; la captura de la sección 10 muestra el nodo de entrada `Recibir fallo automatico`.	Prepara el registro del incidente y lo guarda en Baserow con **`POST`**; si el guardado falla, `Registro no almacenado` deja la ejecución en estado de error.	Registra el incidente; no repara ni reanuda el workflow fallido.

Diagrama 1. Se lee de arriba hacia abajo. La entrada es `Webhook - Reserva`, que recibe **`POST`** `/reserva-hotel`. «Extraer solicitud» organiza los campos y «Llamar a Preparar solicitud» ejecuta el subworkflow de validación. En el rombo «¿Datos completos?», la rama No lleva a «Responder 400 · Solicitud inválida»; la rama Sí llama a «Consultar disponibilidad». En «¿Hay habitación?», la rama Sí pasa por «Crear reserva» (guarda la reserva en Baserow) y «Confirmación» (prepara el mensaje) hasta «Responder 200»; la rama No pasa por «Sin disponibilidad» hasta «Responder 409 · No hay disponibilidad».

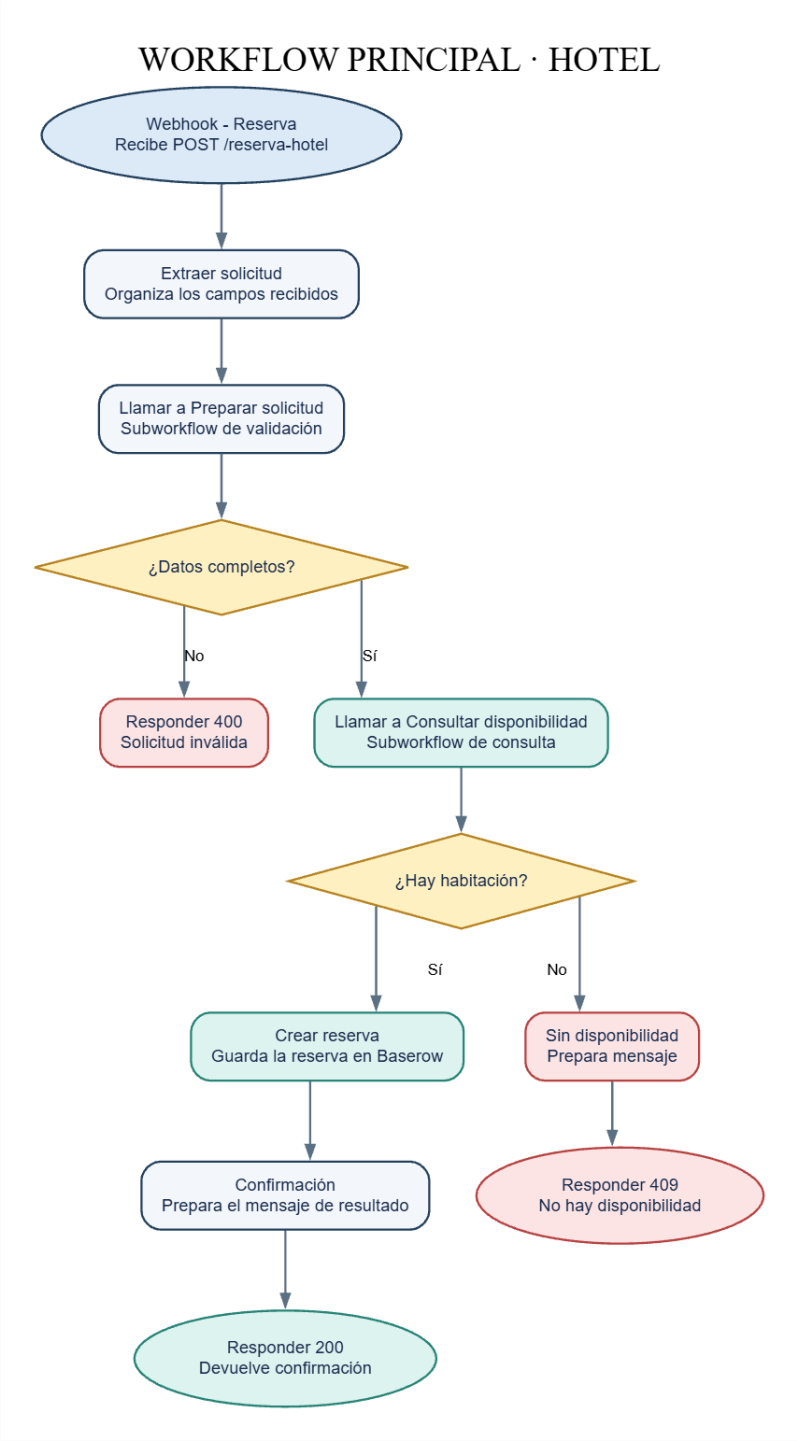


Figura 1. Workflow principal de reservas: recibe la solicitud, valida los datos, consulta disponibilidad y devuelve el resultado.

Diagrama 2. El subworkflow recibe los datos del principal y ejecuta un único bloque, «Validar y normalizar», que limpia espacios, normaliza el email, valida fechas y huéspedes, comprueba campos obligatorios y calcula el número de noches. Termina en una salida estructurada con `valido`, `errores` y los datos normalizados (entrada, salida, huéspedes, tipo y noches).

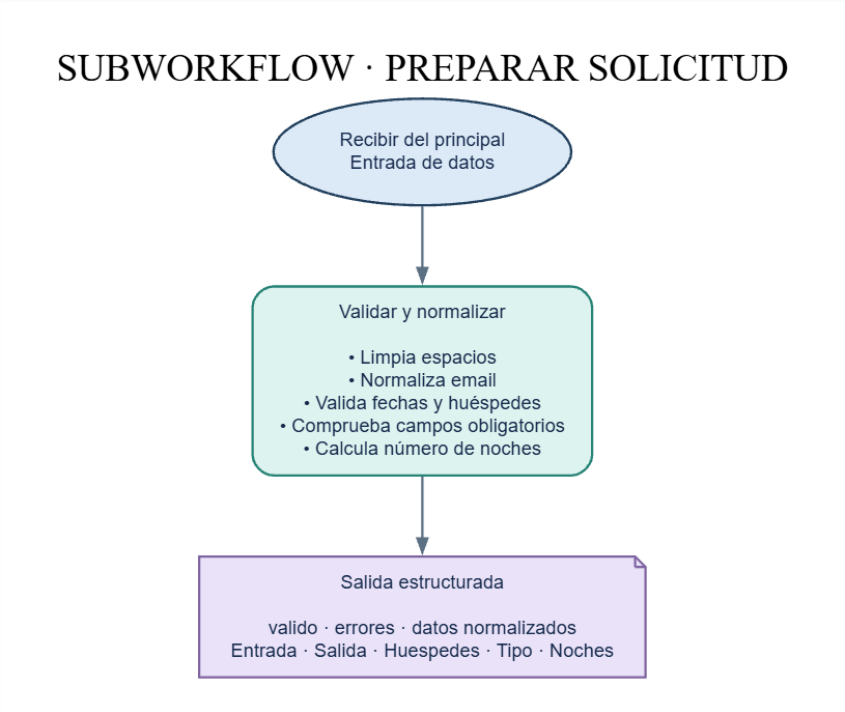


Figura 2. Subworkflow «Preparar solicitud»: normaliza los campos, valida requisitos y calcula el número de noches.

Diagrama 3. El subworkflow recibe tipo, fechas, huéspedes y email. Primero consulta las habitaciones en Baserow (inventario por tipo) y luego las reservas existentes. En «Determinar disponibilidad» busca cruces de fechas, detecta solicitudes duplicadas, selecciona una habitación libre y devuelve el motivo y el resultado. La salida estructurada incluye `disponible`, `duplicada`, el motivo, la habitación y datos de referencia.

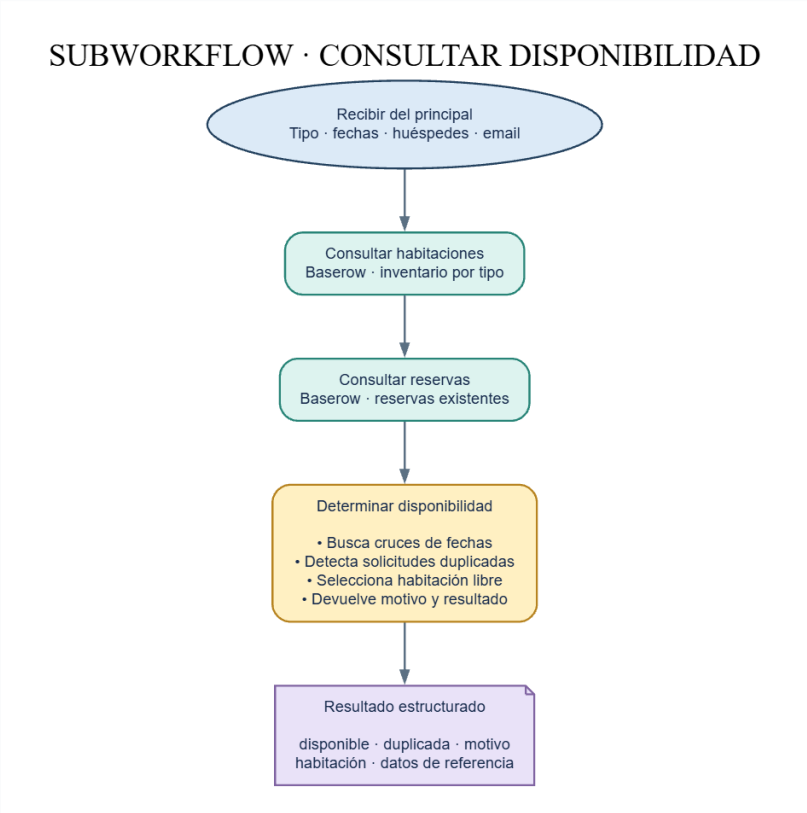


Figura 3. Subworkflow «Consultar disponibilidad»: consulta habitaciones y reservas en Baserow y determina el resultado.

Observación técnica para la entrega: el resumen de apoyo identifica referencias antiguas en los nodos «`Crear reserva`» y «`Confirmación`» del JSON exportado del workflow principal. Antes de presentar el sistema como funcional de extremo a extremo, deben actualizarse esas expresiones para leer los datos devueltos por los subworkflows y repetirse una prueba integral. Google Calendar todavía no está integrado y el workflow de manejo de errores debe asociarse desde la configuración del principal.

9. Validaciones y manejo de errores

Validación de la solicitud

Validar es comprobar que los datos cumplen reglas antes de usarlos. Aquí hay dos niveles: las reglas sobre los datos recibidos (nombre, correo, fechas, huéspedes y tipo), que aplica «Preparar solicitud», y las reglas de negocio (disponibilidad y duplicados), que aplica «Consultar disponibilidad».

Campo o regla	Validación documentada	Consecuencia
Nombre	Obligatorio después de quitar espacios.	Solicitud inválida si está vacío.
Correo	Formato básico de email; se almacena en minúsculas.	Solicitud inválida si no cumple el formato.
Fechas	Formato AAAA-MM-DD y fecha real válida.	Se rechazan formatos incorrectos o fechas inexistentes.
Orden de fechas	La salida debe ser posterior a la entrada.	Se devuelve un error de validación.
Huéspedes	Entero mayor que cero; acepta número o texto numérico.	Se rechazan cero, negativos o valores no enteros.
Tipo de habitación	Texto obligatorio.	Solicitud inválida si está vacío.
Disponibilidad	Comprueba solapamiento con reservas confirmadas.	Busca una habitación libre o devuelve `SIN_DISPONIBILIDAD`.
Duplicados	Compara email, fechas y tipo de habitación en reservas confirmadas.	Devuelve `RESERVA_DUPLICADA` y la referencia de la reserva existente.

Respuestas y fallos

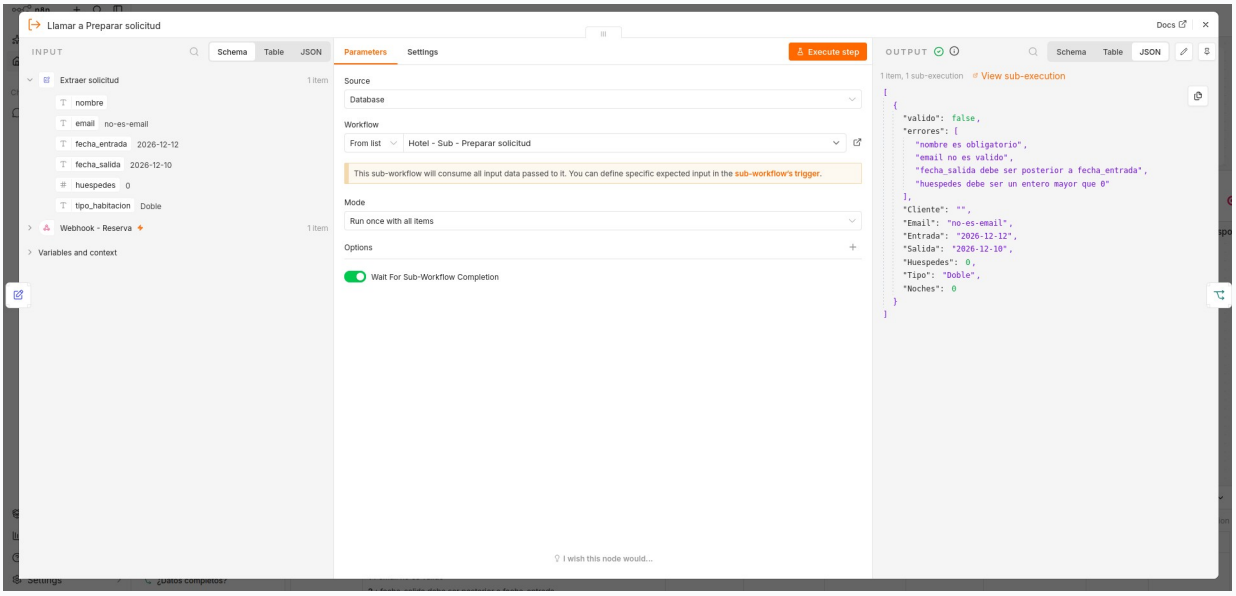
Un código de estado HTTP es el número con que un servicio indica el resultado de una petición: los **2xx** señalan éxito y los **4xx** un problema en la solicitud. Los códigos **400**, **409** y **200** corresponden al workflow principal; el resto los genera el puente local.

Situación	Respuesta / comportamiento descrito
Datos inválidos	El workflow principal tiene una rama para responder HTTP **400**.
No hay disponibilidad	El workflow principal tiene una rama para responder HTTP **409**.
Éxito	El workflow principal incluye una rama de respuesta HTTP **200** y un mensaje de confirmación.
Fallo técnico	El manejador de errores prepara el contexto del fallo y el registro de incidente.
Falla el guardado del incidente	El nodo «`Registro no almacenado`» está diseñado para dejar la ejecución del manejador en estado de error.
Puente no encuentra la ruta o método	**404** `NO_ENCONTRADO`.
Puente recibe demasiadas peticiones	**429** `DEMASIADAS_PETICIONES` tras superar el límite configurado.
El contenido no es JSON	**415** `SOLO_JSON` si el Content-Type no es JSON; **400** `JSON_INVALIDO` si el cuerpo no puede analizarse.

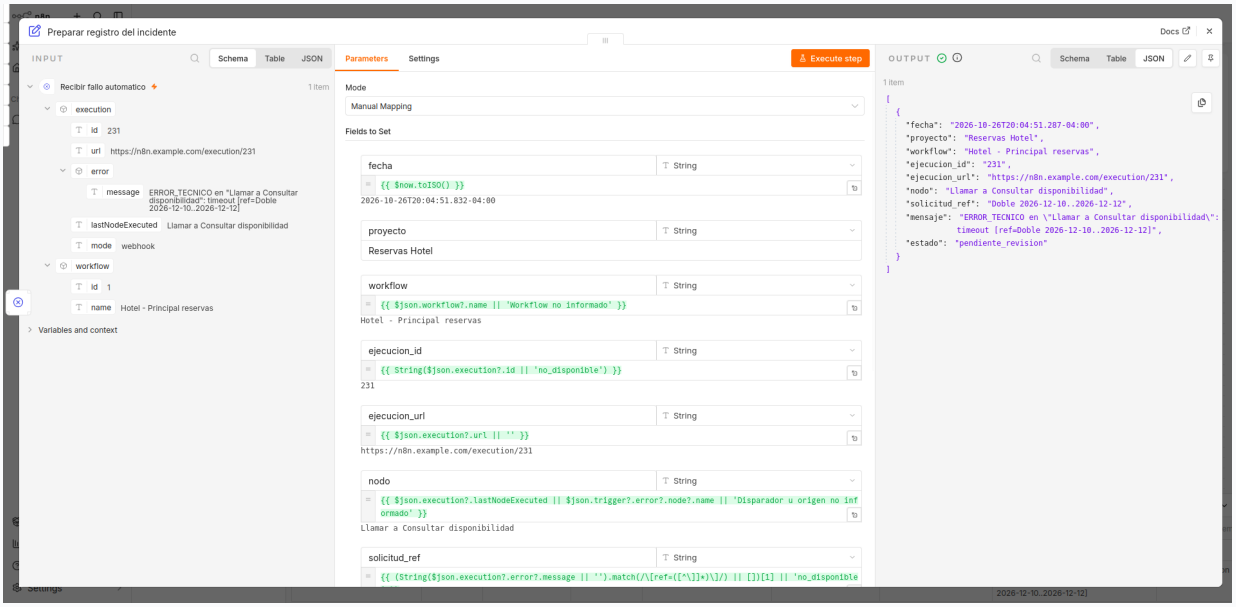
Situación	Respuesta / comportamiento descrito
Cuerpo supera 10 KB	`413` `CUERPO_MUY_GRANDE`.
n8n no responde o excede el tiempo	`502` `N8N_NO_RESPONDE` o `504` `N8N_TARDO_DEMASIADO`.

Qué ocurre en cada caso. Con datos inválidos, el flujo termina antes de consultar Baserow y responde `400`. Sin disponibilidad, responde `409`. Si la solicitud está duplicada, el subworkflow devuelve `RESERVA_DUPLICADA` con la referencia de la reserva existente (en `D-03`, el ID 39) y no propone crear otra; el material no detalla qué respuesta HTTP produce el workflow principal en ese caso, por lo que debe verificarse. La respuesta `503` para fallas técnicas sigue pendiente en el workflow principal, según la nota de alcance de la página de contenido.

Las dos capturas siguientes documentan `P-02`, que envía una solicitud con varios problemas a la vez, y `M-01`, que comprueba cómo un fallo simulado se convierte en un registro de incidente.



Evidencia de preparación con entrada no válida. La bitácora reporta cuatro errores detectados en conjunto.



Evidencia M-01: mapeo de datos simulados al registro de incidente.

Diferencia entre error de negocio y error técnico

Una solicitud inválida o una reserva duplicada puede terminar con una decisión controlada del workflow. Un error técnico es una falla durante una operación, por ejemplo, un servicio externo que no responde. El manejador de errores está destinado a registrar este segundo tipo de situación.

10. Pruebas, resultados y evidencias

Las bitácoras describen pruebas manuales con datos ficticios y capturas. Los resultados deben interpretarse según el alcance de cada prueba: las de preparación validan transformaciones y reglas; las de disponibilidad verifican decisiones frente a las tablas; la prueba del manejador de errores valida el mapeo de un fallo simulado.

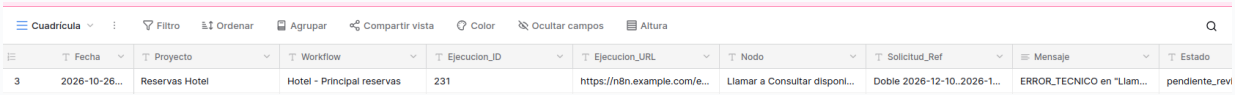
ID	Qué se probó	Resultado registrado	Alcance
`P-01`	Solicitud válida con datos sucios.	**Correcto**: limpia nombre, convierte correo a minúsculas y huéspedes a número; calcula 2 noches.	Prueba manual de preparación.
`P-02`	Solicitud con cuatro problemas a la vez.	**Correcto**: detecta nombre vacío, email inválido, salida anterior a entrada y huéspedes igual a cero.	Prueba manual de validación; no prueba cada regla por separado.
`D-01`	Fechas libres.	**Correcto**: habitación 102.	Consulta de disponibilidad de solo lectura.
`D-02`	Cruce con reservas existentes.	**Correcto**: descarta habitación 102 y elige la 201.	Consulta de disponibilidad de solo lectura.
`D-03`	Solicitud repetida.	**Correcto**: identifica la reserva existente ID 39.	Consulta de disponibilidad de solo lectura.
`D-04`	Tipo de habitación inexistente.	**Correcto**: `SIN_DISPONIBILIDAD`.	Consulta de disponibilidad de solo lectura.
`D-05`	Entrada el mismo día que sale otro huésped.	**Correcto**: habitación 102 disponible.	Comprueba la regla de límites de fechas.
`M-01`	Mapeo de incidente con datos simulados.	**Correcto**: genera los campos de registro y extrae la referencia de solicitud.	No demuestra activación automática ante un fallo real.
`M-02`	Escritura persistente en la tabla Incidencias.	**La bitácora marca esta prueba como pendiente.**	La captura incluida parece mostrar una fila, pero la bitácora no registra el resultado como completado; debe verificarse.
`M-03`	Falla el propio registro del incidente.	**La bitácora marca esta prueba como pendiente.**	La imagen incluida muestra el diseño del workflow, no acredita por sí sola una ejecución fallida.

Para leer la tabla con rapidez, los resultados se agrupan por estado:

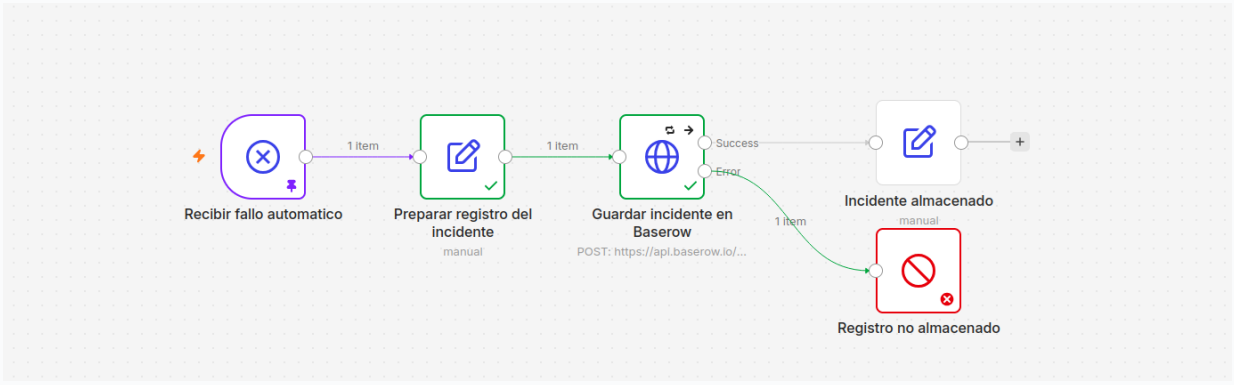
Estado	Pruebas	Qué significa
Satisfactorias	`P-01`, `P-02`, `D-01` a `D-05`, `M-01`	La bitácora registra «Correcto». Las transformaciones, reglas y decisiones funcionan a nivel de nodo con datos ficticios; no son pruebas integrales.
Pendientes	`M-02`, `M-03`	La bitácora las marca como pendientes: no se puede afirmar que la escritura persistente de incidencias ni la falla del propio registro estén comprobadas.
Requieren verificación	Capturas de `M-02` y `M-03`	La captura de la tabla Incidencias debe confirmarse contra `M-02`, y la imagen del diseño del workflow no acredita una ejecución de `M-03`.

Qué se puede concluir

- Se observó que la preparación transforma una entrada válida al formato normalizado esperado.
- Se observó que la validación devuelve varios errores cuando una solicitud contiene más de un problema.
- Se observó que el subworkflow de disponibilidad distingue entre fechas libres, cruces, duplicados y ausencia del tipo solicitado.
- Se observó el mapeo de un error simulado a un objeto de incidente con referencia de solicitud.
- No se debe afirmar que se ha probado una transacción integral real ni que el registro persistente de incidencias está completado sin verificar los puntos pendientes.



Captura incluida en el ZIP relacionada con la tabla Incidencias; la bitácora todavía marca M-02 como pendiente, por lo que se requiere confirmar su correspondencia y estado.



Captura del diseño del workflow de manejo de errores; no se toma como evidencia concluyente de que M-03 haya sido ejecutada.

11. Limitaciones y mejoras futuras

Las limitaciones y pendientes siguientes provienen de las bitácoras y del material de apoyo; se conservan sin atenuarlas.

Limitación / pendiente	Mejora recomendada
No hay evidencia de una prueba completa desde petición HTTP real hasta respuesta y persistencia final.	Ejecutar un caso de punta a punta con datos ficticios, guardar la respuesta HTTP y verificar el registro final en Baserow.
`M-02` y `M-03` figuran como pendientes en la bitácora, aunque el ZIP contiene nodos y capturas relacionados.	Actualizar la bitácora tras verificar la ejecución real y asociar cada captura al resultado exacto.
No se incluyen métricas cuantitativas de tiempo o errores.	Medir el tiempo de atención antes y después y registrar resultados de una muestra definida.
El manejador registra, pero no repara ni reanuda el proceso.	Como mejora futura, incorporar alertas y un procedimiento de revisión; cualquier reintento debe diseñarse con controles para evitar duplicados.
No se adjunta el enlace del repositorio GitHub.	Agregar la URL del repositorio correcto antes de exportar y entregar el documento.
Las capturas pueden contener información de tablas externas.	Usar únicamente datos ficticios y ocultar correos o identificadores personales antes de compartir evidencias.

Otros pendientes señalados en este documento

Además de la tabla anterior, el documento menciona estos puntos abiertos en otras secciones:

- Referencias antiguas en los nodos `Crear reserva` y `Confirmación` del workflow principal, que deben actualizarse y comprobarse con una prueba integral (sección 8).
- Google Calendar todavía no está integrado (sección 8).
- El workflow de manejo de errores debe asociarse desde la configuración del principal (sección 8).
- La respuesta **503** para fallas técnicas sigue pendiente en el workflow principal, y se recomienda rotar las claves expuestas durante las pruebas (nota de alcance de la página de contenido).

12. Repositorio y estado de la entrega

Enlace de GitHub pendiente

El ZIP recibido no contiene una URL de GitHub ni una referencia inequívoca al repositorio remoto. No se inventa un enlace. Antes de entregar, insertar aquí la dirección real del repositorio: [AGREGAR URL DE GITHUB].

El archivo ZIP sí contiene cuatro exportaciones JSON de workflows, tres documentos Markdown de pruebas, capturas PNG y un archivo puente de laboratorio dentro de un ZIP anidado. Estos materiales sustentan la descripción técnica incluida en este documento.

Glosario

Término	Explicación
API	Interfaz que permite que un sistema solicite o envíe datos a otro mediante reglas definidas.
Webhook	Punto de entrada HTTP que recibe datos enviados por otro sistema y activa un workflow.
Subworkflow	Workflow auxiliar llamado desde otro workflow para ejecutar una tarea delimitada.
HTTP POST	Método usado para enviar datos al servicio, por ejemplo, una nueva solicitud de reserva.
HTTP GET	Método usado para consultar datos sin solicitar su creación o modificación.
JSON	Formato de texto estructurado para representar datos como objetos y listas.
Baserow	Herramienta de tablas de datos usada aquí para consultar habitaciones y reservas y para el registro previsto de incidencias.
n8n	Herramienta de automatización que conecta pasos y servicios mediante workflows.
Cruce de fechas	Situación en la que dos reservas ocupan periodos que se solapan para una misma habitación.
Incidencia	Registro técnico que describe una ejecución fallida para facilitar su diagnóstico.
Nodo	Paso individual de un workflow de n8n que realiza una tarea concreta, como recibir datos, consultar una tabla o responder.
Código de estado HTTP	Número que indica el resultado de una petición HTTP; por ejemplo, 200 señala éxito y 400 una solicitud incorrecta.
Normalizar	Llevar un dato a un formato uniforme, por ejemplo, convertir un correo a minúsculas o un número escrito como texto a número.

Nota de elaboración

Este documento se elaboró a partir de los archivos JSON, las bitácoras Markdown y las capturas contenidos en BitacoraYRepo.zip. Se han separado los resultados observados de los comportamientos configurados y de los pendientes, para evitar presentar como completadas pruebas que la propia bitácora marca como no realizadas.